

UNIVERSITE PARIS CITE
UFR MATHÉMATIQUES ET INFORMATIQUE

Comparaison des grands modèles de
langage open-source (LLMs) :
LLaMA, Mistral, DeepSeek.

Master 1 DCI

Amar MERABTI – Lynda HAMMOUCHE

Encadré par Mohamed BOUADI

Année universitaire 2024 – 2025

Table Des Matières :

1.Introduction :	1
2. État de l'art :	1
2.1 LLMs :	1
2.1.1 Définition :	1
2.1.2 Architectures de modèles LLM basées sur les Transformers :	2
Embeddings et positional embeddings :	2
Self-Attention :	3
2.1.3 Les différents types d'architectures :	3
2.2 LLaMA 7B :	4
2.3 Mistral 7B :	4
2.4 DeepSeek R1-Distill-Qwen-7B:	5
2.5 Comparaison des performances :	6
2.5.1 Benchmarks:	6
Raisonnement : HellaSwag.....	6
Génération de code : HumanEval.....	7
2.5.2 Comparaison :	7
2.6 Conclusion :	8
3. Contribution :	9
3.1 Outils et environnement de développement :	9
3.2 Evaluation des LLMs	10
3.2.1 Math GSM8K Raisonnement arithmétique :	10
3.2.2 Raisonnement : HellaSwag.....	14
3.2.3 Génération de code : HumanEval :	17
4. Conclusion.....	23
5. Bibliographie :	25

1.Introduction :

Les Large Language Models (LLMs) ont révolutionné le traitement du langage naturel (NLP) et plus largement l'intelligence artificielle. Grâce à des avancées majeures en apprentissage profond, ces modèles sont aujourd'hui capables de générer du texte cohérent, de répondre à des questions complexes et d'assister dans une multitude de tâches, allant de la traduction automatique à la programmation. L'essor des LLMs a été largement influencé par l'article fondateur de Google, "Attention Is All You Need", publié en 2017. Cet article a introduit le mécanisme de l'attention et les transformeurs, qui ont permis d'améliorer significativement la qualité des modèles de traitement du langage en remplaçant les architectures traditionnelles comme les RNNs et les LSTMs.

Dans ce projet, nous nous intéressons à la comparaison de trois LLMs récents et open-source : **LLaMA** (Meta), **Mistral** (Mistral AI) et **DeepSeek** (DeepSeek AI). Ces modèles se distinguent chacun par leur architecture, leur efficacité et leurs performances sur des tâches variées.

Dans un contexte où les LLMs sont de plus en plus intégrés dans des applications concrètes ; assistants virtuels, outils de codage, systèmes de recommandation ou agents autonomes, il devient essentiel de disposer de points de comparaison clairs et objectifs pour orienter les choix technologiques. Notre étude s'appuiera donc sur une série de benchmarks spécifiques pour évaluer les performances de ces trois modèles, dans le but de mettre en évidence leurs points forts et limites respectives selon les tâches.

2. État de l'art :

Dans cette partie, nous présentons les bases des modèles de langage de grande taille ainsi qu'un aperçu des modèles étudiés et des benchmarks utilisés pour les comparer.

2.1 LLMs :

2.1.1 Définition :

Un grand modèle linguistique (Large Language Model, LLM) est un type de programme d'intelligence artificielle capable, entre autres tâches, de reconnaître et de générer du texte. Les LLMs sont entraînés sur de vastes ensembles de données, d'où l'emploi du terme « large » dans la dénomination anglaise. Ils s'appuient sur l'apprentissage automatique, plus spécifiquement sur un type de réseau neuronal "modèle transformeur"

2.1.2 Architectures de modèles LLM basées sur les Transformers :

Les architectures Transformer ont largement dominé le domaine des modèles de langage de grande taille (LLM), grâce à leur efficacité à traiter de longues séquences et à capturer des relations contextuelles complexes. Ce modèle a inspiré de nombreuses variantes, qui offrent des améliorations spécifiques pour optimiser les performances ou adapter les modèles à des tâches particulières.

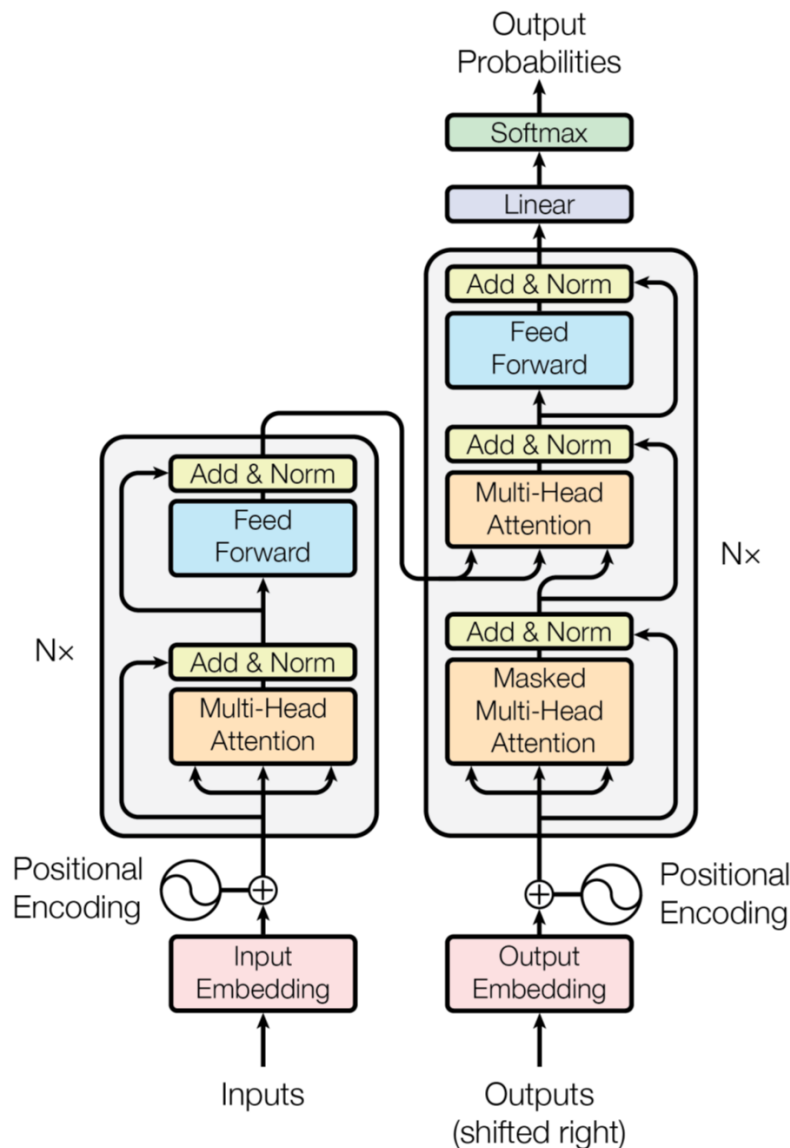


Figure 1: Architecture du Transformer source: Attention Is All You Need.[7]

Embeddings et positional embeddings:

Le texte d'entrée est segmenté en unités plus petites (tokens), telles que des mots ou des sous-mots, et chaque segment est intégré dans une représentation vectorielle continue. Cette étape d'intégration capture les informations sémantiques et syntaxiques de l'entrée. Le codage positionnel est ajouté aux intégrations d'entrée pour fournir des informations sur les positions des tokens.

Self-Attention :

L'élément central des modèles Transformers repose sur le mécanisme de "Self-Attention". C'est ce mécanisme qui leur permet d'avoir une bonne compréhension du langage. Son principe ressemble un peu à ce qui se passe dans notre cerveau quand on lit une phrase. On analyse « simultanément » tous les mots de la phrase et les relations qui les lient, sans se focaliser sur un seul mot uniquement. Au lieu de considérer les tokens un par un, le modèle va considérer tous les tokens de la phrase simultanément et pour chaque token, il va accorder des niveaux d'importance (on parle de poids) aux autres tokens de la phrase.

- **Self-Attention dans l'Encodeur** : Dans l'encodeur, le self-attention est bidirectionnel, c'est-à-dire que chaque token peut attendre tous les autres tokens de la séquence, qu'ils soient avant ou après lui. Cela permet au modèle de capturer les dépendances globales dès le début, essentiel pour comprendre le sens général du texte.
- **Self-Attention dans le Décodeur** : Dans le décodeur, le self-attention est causal (ou masqué) : un token ne peut considérer que les tokens précédents et pas ceux qui suivent. Ce masquage garantit que la génération du texte se fait de manière autoregressive, c'est-à-dire un token après l'autre.

2.1.3 Les différents types d'architectures :

Il existe plusieurs types d'architectures de modèles Transformers :

- **Encoder-Decoder** : Cette architecture comprend une série d'encodeurs suivie d'une série de décodeurs. C'est le modèle original introduit par Google pour la traduction. Elle est également adaptée à des tâches comme la summarisation.
- **Encoder-Only** : Ici, seuls des encodeurs sont empilés les uns sur les autres, sans décodeur. Ces architectures transforment l'entrée en représentations contextuelles riches sans générer directement de nouvelles séquences. Elles sont particulièrement efficaces pour des tâches comme l'analyse de sentiments et la classification de texte.
- **Decoder-Only** : Dans cette architecture, seuls des décodeurs sont empilés, sans encodeur. Elle est idéale pour les modèles génératifs comme GPT ou LLaMA, utilisés pour la génération de texte.

2.2 LLaMA 7B :

LLaMA [8] est une famille de modèles de langage développée par Meta AI. L'objectif de cette initiative est de fournir des modèles puissants et accessibles à la communauté scientifique, tout en s'appuyant exclusivement sur des jeux de données ouverts.

Architecture : LLaMA 7B est basé sur l'architecture Transformer de type Decoder-Only, avec des améliorations spécifiques pour l'efficacité et la performance. Il adopte plusieurs optimisations par rapport aux modèles standards :

- Pré-normalisation : normalisation en entrée des sous-couches du transformer pour une meilleure stabilité.
- SwiGLU : fonction d'activation améliorant les performances.
- Rotary Embeddings (RoPE) : supprime les embeddings de position absolue pour une meilleure généralisation.

Spécificités :

- Taille : 7 milliards de paramètres.
- Entraînement sur des données publiques : contrairement à d'autres modèles, LLaMA est entraîné sur des données accessibles au public, rendant sa publication plus ouverte.

Entraînement : LLaMA 2 7B a été entraîné sur un corpus massif de données multilingues de 2 trillions de tokens, issu de sources diverses (Common Crawl, C4, GitHub, Wikipedia, etc.). L'entraînement a été réalisé avec l'optimiseur AdamW, et a nécessité des milliers de GPU A100.

2.3 Mistral 7B :

Mistral 7B [1] est développé par Mistral AI, une entreprise française spécialisée dans les modèles de langage de grande taille. Elle se positionne comme un acteur clé en proposant des modèles ouverts et optimisés pour l'inférence.

Architecture : Mistral 7B repose également sur une architecture Transformer de type Decoder-Only, avec plusieurs optimisations avancées :

- Grouped-Query Attention (GQA) : accélère l'inférence et réduit la consommation mémoire.
- Sliding Window Attention (SWA) : permet de traiter efficacement des séquences plus longues avec un coût réduit.
- Rolling Buffer Cache : optimise la gestion des caches et réduit la consommation mémoire en inférence.

Spécificités :

- Taille : 7 milliards de paramètres.
- Facilité de fine-tuning : conçu pour être facilement adapté à différentes tâches spécifiques.

Entraînement : Mistral 7B a été entraîné sur des ensembles de données diversifiés, sélectionnés pour améliorer la compréhension et la cohérence du texte généré. L'entraînement a été effectué avec une infrastructure de GPU haute performance, permettant d'optimiser les coûts et le temps d'entraînement sans compromettre la qualité du modèle.

2.4 DeepSeek R1-Distill-Qwen-7B:

DeepSeek R1-Distill-Qwen[6] est un modèle développé par DeepSeek AI, une entreprise chinoise spécialisée dans l'intelligence artificielle générative. Cette version distillée repose sur le modèle DeepSeek-R1 7B, lui-même inspiré des architectures LLaMA et Qwen. Elle vise à combiner l'efficacité de l'inférence avec la qualité des réponses du modèle Qwen, tout en restant open-source et accessible pour les usages grand public ou professionnels.

Architecture :

DeepSeek-R1-Distill-Qwen-7B repose sur une architecture Transformer de type Decoder-Only, avec plusieurs optimisations avancées visant à améliorer l'efficacité en inférence et la qualité des réponses :

- **MoE Structure (Mixture of Experts)** : active dynamiquement certains blocs du modèle, ce qui permet de réduire les coûts de calcul tout en conservant des performances élevées.
- **Multi-Head Latent Attention (MLA)** : enrichit le mécanisme d'attention pour mieux capter les relations complexes à long terme dans les séquences.
- **Multi-Token Prediction (MTP)** : autorise la prédiction de plusieurs tokens simultanément, accélérant l'inférence.

Spécificités :

- **Taille** : 7 milliards de paramètres.
- **Origine** : distillation supervisée à partir de DeepSeek-R1 et du modèle Qwen.
- **Orientation raisonnement** : optimisé pour les tâches nécessitant des capacités de raisonnement.

- **Efficacité** : conçu pour offrir un bon compromis entre performance et légèreté, idéal pour l'inférence à moindre coût

Entraînement : DeepSeek-R1-Distill-Qwen-7B a été affiné à partir du modèle open source Qwen, en utilisant 800 000 échantillons générés par DeepSeek-R1. Ce processus de distillation a permis de transférer les capacités de raisonnement du modèle principal vers des modèles plus petits, tout en maintenant une grande efficacité en inférence. L'entraînement s'est concentré sur la fine-tuning supervisée (SFT), sans passer par une phase de renforcement (RLHF), dans le but de démontrer l'efficacité de la distillation seule.

2.5 Comparaison des performances :

Dans ce qui suit on présentera une comparaison des trois modèles sur différents benchmarks.

2.5.1 Benchmarks :

Les benchmarks des LLMs sont des tests standardisés qui évaluent la performance des modèles sur diverses tâches. Ils vérifient généralement si le modèle peut produire la réponse correcte attendue à un input donné.

Dans ce qui suit, nous présenterons les benchmarks utilisés dans notre étude pour tester et comparer les performances des modèles.

GSM8K Raisonnement arithmétique :

L'ensemble de données GSM8K, publié en 2021 est l'un des points de référence bien connus pour évaluer les capacités en mathématiques dans les LLM, il contient des problèmes de mathématiques de niveau intermédiaire, qui nécessitent quelques étapes à résoudre, et les solutions impliquent principalement d'effectuer une séquence de calculs élémentaires.

Raisonnement : HellaSwag

Il s'agit d'un benchmark conçu pour évaluer l'inférence en langage naturel basée sur le bon sens. Il demande au modèle de prédire la fin la plus probable d'une phrase. HellaSwag est structuré comme une tâche à choix multiple. Les réponses incluent des options adversariales : des réponses incorrectes générées par une machine, mais qui semblent plausibles et nécessitent un raisonnement approfondi pour être écartées.

Génération de code : HumanEval

Le jeu de données HumanEval comprend 164 problèmes de programmation rédigés manuellement pour évaluer la compréhension du langage, les algorithmes et les mathématiques de base, certains étant comparables à des questions d'entretien technique en développement logiciel.

2.5.2 Comparaison :

Plusieurs comparaisons de performances ont été effectuées entre Mistral 7B et LLaMA 2 7B sur différents benchmarks. Les résultats montrent que Mistral 7B surpasse systématiquement LLaMA 2 7B, notamment dans les tâches de codage, de mathématiques et de raisonnement, où ses performances se démarquent nettement.

Model	Modality	MMLU	HellaSwag	WinoG	PIQA	Arc-e	Arc-c	NQ	TriviaQA	HumanEval	MBPP	MATH	GSM8K
LLaMA 2 7B	Pretrained	44.4%	77.1%	69.5%	77.9%	68.7%	43.2%	24.7%	63.8%	11.6%	26.1%	3.9%	16.0%
LLaMA 2 13B	Pretrained	55.6%	80.7%	72.9%	80.8%	75.2%	48.8%	29.0%	69.6%	18.9%	35.4%	6.0%	34.3%
Code-Llama 7B	Finetuned	36.9%	62.9%	62.3%	72.8%	59.4%	34.5%	11.0%	34.9%	31.1%	52.5%	5.2%	20.8%
Mistral 7B	Pretrained	60.1%	81.3%	75.3%	83.0%	80.0%	55.5%	28.8%	69.9%	30.5%	47.5%	13.1%	52.2%

Tableau 1 : Comparaison de Mistral 7B avec Llama. Source : Mistral 7b [1]

En ce qui concerne DeepSeek-R1 7B, le modèle a démontré d'excellentes performances en mathématiques, notamment sur le benchmark Math-500, reconnu pour être significativement plus exigeant que GSM8K. En génération de code, ses résultats sur LiveCodeBench — un benchmark plus complexe et réaliste que HumanEval — sont corrects, bien qu'un peu en retrait par rapport aux meilleurs modèles. Cette tendance positionne néanmoins DeepSeek-R1 7B légèrement au-dessus de Mistral 7B en termes de performance globale. Par ailleurs, ses performances robustes sur les tâches de raisonnement scientifique suggèrent une bonne capacité à traiter des contenus spécialisés, renforçant son profil comme un modèle polyvalent et compétitif.

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9	1316
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633

Tableau 2 : Comparaison des modèles distillés DeepSeek-R1 avec d'autres modèles sur des benchmarks liés au raisonnement. Source: DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning [6].

2.6 Conclusion :

L'examen de l'état de l'art des modèles de langage de grande taille (LLMs) met en lumière les avancées majeures en matière d'architecture et de performance. Tous les modèles étudiés – LLaMA 2 7B, Mistral 7B, et DeepSeek-R1-Distill-Qwen-7B – s'appuient sur l'architecture Transformer de type Decoder-Only, optimisée par des techniques spécifiques visant à améliorer l'efficacité, la généralisation et la rapidité d'inférence.

Chaque modèle intègre des choix techniques qui influencent directement ses résultats. LLaMA 2 adopte une structure sobre et robuste avec rotary embeddings et normalisation en entrée. Mistral 7B se distingue par des mécanismes innovants tels que le Grouped-Query Attention et le Sliding Window Attention, lui permettant d'exceller sur les séquences longues. DeepSeek-R1 7B, quant à lui, tire parti de la distillation supervisée, de la structure MoE et de la prédiction multi-token pour équilibrer performance et efficacité.

L'évaluation comparative sur des benchmarks standards révèle des profils de performance différenciés. Mistral 7B s'impose comme le plus régulier, avec des résultats très solides en codage, mathématiques et raisonnement. DeepSeek-R1 7B, démontre une forte aptitude en raisonnement mathématique et logique.

Ces observations suggèrent que certains modèles sont particulièrement adaptés à des types de tâches spécifiques, en raison de leur conception et de leurs choix d'entraînement.

La section suivante se consacre à une évaluation empirique approfondie de ces modèles sur des benchmarks ciblés, dans le but de consolider cette analyse comparative.

3. Contribution :

Dans cette partie, nous décrivons l'environnement de travail que nous avons mis en place, ainsi que la méthodologie utilisée pour évaluer les LLMs sur les trois benchmarks.

3.1 Outils et environnement de développement :

Pour l'environnement matériel et logiciel, nous avons utilisé l'espace de travail Kaggle pour exécuter nos algorithmes. Les détails sont présentés ci-dessous.

1. Matériel

- **GPU** : 4 × NVIDIA L4 avec 23034 MiB de VRAM chacune ; version du pilote 560.35.03 ; CUDA 12.6 (via nvidia-smi) ; matériel fourni par Kaggle.
- **CPU** : Processeur Intel(R) Xeon(R) @ 2,20 GHz, 24 cœurs physiques / 48 threads (via lscpu) ; fonctionnant dans l'environnement virtualisé de Kaggle.
- **Mémoire** : Environ 170+ Go de RAM DDR4 au total (via free -h) ; allouée par Kaggle.

2. Software

L'environnement logiciel utilisé pour les expériences était basé sur l'espace de travail Kaggle, avec les outils et bibliothèques suivants :

- **Python** : Langage principal utilisé pour le développement des scripts et l'exécution des expériences. La version utilisée était Python 3.10, fournie par l'environnement Kaggle.
- **Pandas** : Bibliothèque de manipulation de données en Python, utilisée pour le traitement des jeux de données tabulaires (chargement, nettoyage, filtrage, etc.).
- **Scikit-learn (sklearn)** : Bibliothèque de référence pour l'apprentissage automatique classique, utilisée notamment pour l'évaluation avec les ses métriques prédéfinies.
- **Matplotlib** : Bibliothèque de visualisation en 2D, utilisée pour tracer des graphiques et représenter les résultats de manière claire (histogrammes, courbes, etc.).
- **Datasets (Hugging Face)** : Bibliothèque permettant de charger facilement des jeux de données standards pour l'entraînement et l'évaluation, en particulier dans le domaine du traitement du langage naturel (NLP) et les LLMs.
- **Ollama** : Interface locale pour exécuter des modèles de langage open-source de manière efficace, utilisée ici pour interagir avec les LLMs testés dans l'étude.
- **Kaggle** : Plateforme de science des données utilisée à la fois pour le stockage des jeux de données, la mise à disposition d'un environnement d'exécution GPU, et le partage de notebooks reproductibles.

Métriques d'évaluation des modèles :

Pour l'évaluation des trois modèles, nous avons utilisé deux métriques principales. La première est l'accuracy, qui mesure le pourcentage de solutions générées par le modèle et jugées correctes par rapport aux tests fournis. Elle donne une indication globale du taux de réussite sans prendre en compte le type d'échec. La seconde est le score F1, une moyenne harmonique entre la précision et le rappel, permettant une évaluation plus fine en tenant compte à la fois des vraies prédictions positives et des erreurs. Cette métrique est particulièrement utile pour évaluer la performance des modèles sur des tâches avec des sorties variables ou déséquilibrées.

Des détails supplémentaires sur la manière exacte dont ces métriques ont été calculées seront expliqués dans chaque section de benchmark, en fonction des spécificités de l'évaluation appliquée à chaque modèle.

3.2 Evaluation des LLMs

3.2.1 Math GSM8K Raisonnement arithmétique :

1.Description du Dataset :

Le dataset utilisé pour cette évaluation est **GSM8K [5]**. Il contient des problèmes et des réponses de mathématiques destinées aux élèves du primaire et du collège. Ce benchmark, conçu par des chercheurs d'OpenAI, vise à tester la capacité des LLMs à répondre librement en réalisant des opérations arithmétiques simples (addition, multiplication, soustraction, division) sur une séquence de deux à huit étapes.

2. Méthodologie :

Le principal défi ici est de faire en sorte que le modèle respecte un format de réponse précis, afin de pouvoir extraire automatiquement sa réponse et la comparer à la réponse correcte. En nous appuyant sur l'article de Mistral 7B, nous avons adopté une approche d'évaluation en 8-shots.

- **Approche 8-Shot** : Dans cette méthode, on fournit au modèle 8 exemples de questions-réponses, présentés selon un format précis, extraits des données d'entraînement du dataset GSM8K. Cela permet au modèle de mieux comprendre la structure attendue et d'améliorer la qualité de ses réponses.
- **Préparation des Données** : Nous avons sélectionné 500 questions issues de la partie test du dataset GSM8K pour évaluer nos modèles.
- **Interaction avec le système** :
 - Un prompt système a été défini pour guider précisément les modèles sur leur rôle et les exigences strictes concernant le format de sortie. Les modèles étaient instruits pour agir comme des solveurs mathématiques respectant des consignes très strictes de formatage. La réponse finale

devait être un seul nombre, placé à la fin de la réponse et précédé de la séquence `\n####`. Les modèles devaient suivre ce format à l'identique, comme illustré dans les exemples "shots" fournis.

- Pour chaque test soumis au modèle, un prompt utilisateur a été construit, présentant la question à résoudre dans le format attendu.
- **Extraction des Réponses :** Pour récupérer la réponse finale fournie par le modèle, nous utilisons une fonction d'extraction basée sur une expression régulière. Cette fonction recherche spécifiquement un marqueur `####` suivi du nombre attendu, ce qui correspond au format strict imposé dans le prompt. Si ce format précis n'est pas respecté, la fonction tente de récupérer le dernier nombre présent dans la réponse, Si aucune valeur numérique n'est détectée, la fonction retourne `None`.

3. Calcul des Métriques de Performance

Pour chaque tâche, une prédiction est considérée comme correcte si la réponse extraite du modèle correspond exactement à la solution de référence présente dans le jeu de données. Les réponses mal structurées, entraînant une valeur `NaN` lors de l'extraction, sont considérées comme incorrectes. L'accuracy est ensuite calculée en tenant compte de ces cas invalides.

Pour le calcul du F1-score, une colonne binaire a été ajoutée, indiquant `True` si la réponse est correcte, et `False` sinon. Le score F1 est ensuite calculé à l'aide de la fonction prédéfinie `f1_score` de la bibliothèque `sklearn.metrics`, en comparant les cette colonne à un vecteur de vérités de terrain composé exclusivement de valeurs.

4. Résultats et Analyses :

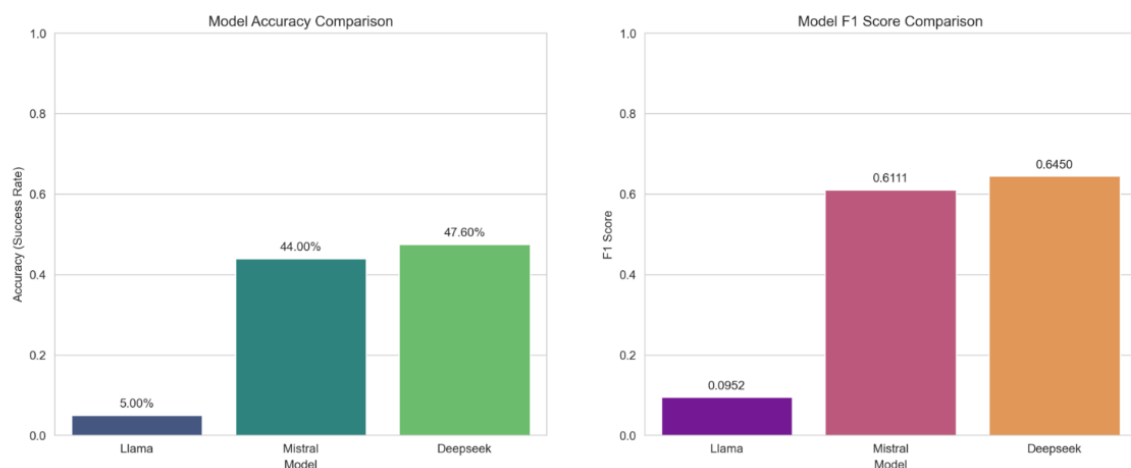


Figure 2 : Résultats de l'évaluation de GSM8K.

On peut voir dans ces résultats que pour :

- **Accuracy :**
 - DeepSeek a obtenu la meilleure accuracy avec environ 47.6%
 - Mistral, proche de DeepSeek, affiche une accuracy d'environ 44 %.
 - Llama a montré une performance nettement inférieure, avec une accuracy d'environ 5%.
- **F1-Score :**
 - Les tendances du F1-Score suivent le même ordre de l'accuracy.
 - DeepSeek étant toujours le meilleur avec un F1-score de 0.64.
 - Mistral le suit de près avec un F1-Score d'environ 0,61
 - Enfin, LLaMA affiche, sans surprise, le F1-Score le plus bas à 0,09.

Un autre résultat intéressant de cette évaluation concerne le nombre de réponses valides fournies par chaque modèle.

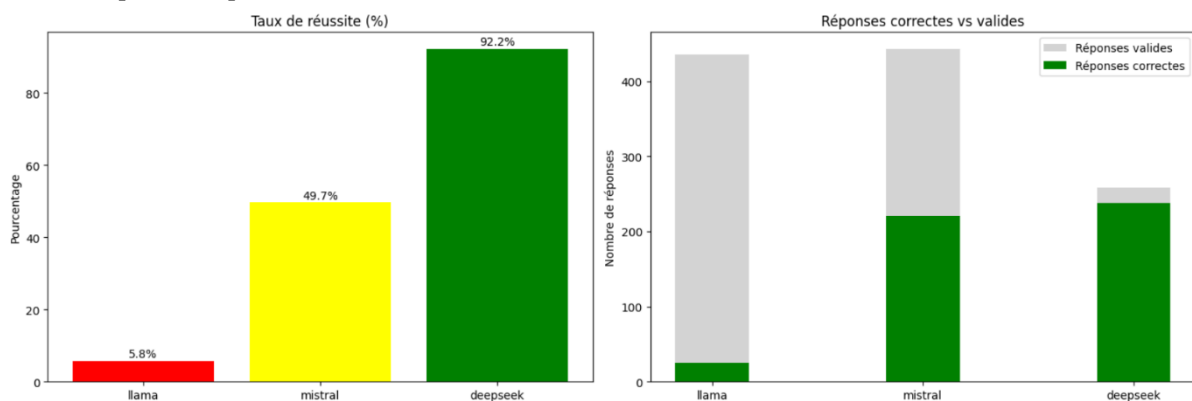


Figure 3 : GSM8K réponses valides VS réponses correctes.

On peut voir que :

- DeepSeek a généré le moins de réponses valides (258 sur 500), mais avec un taux de réponses correctes très élevé, atteignant 92 %.
- Mistral a produit le plus grand nombre de réponses valides (443 sur 500), avec environ 43 % de bonnes réponses.
- LLaMA a fourni un nombre de réponses valides proche de celui de Mistral (435 sur 500), mais avec un pourcentage de bonnes réponses nettement plus faible, seulement 5,8 %.

Efficacité en inférence :

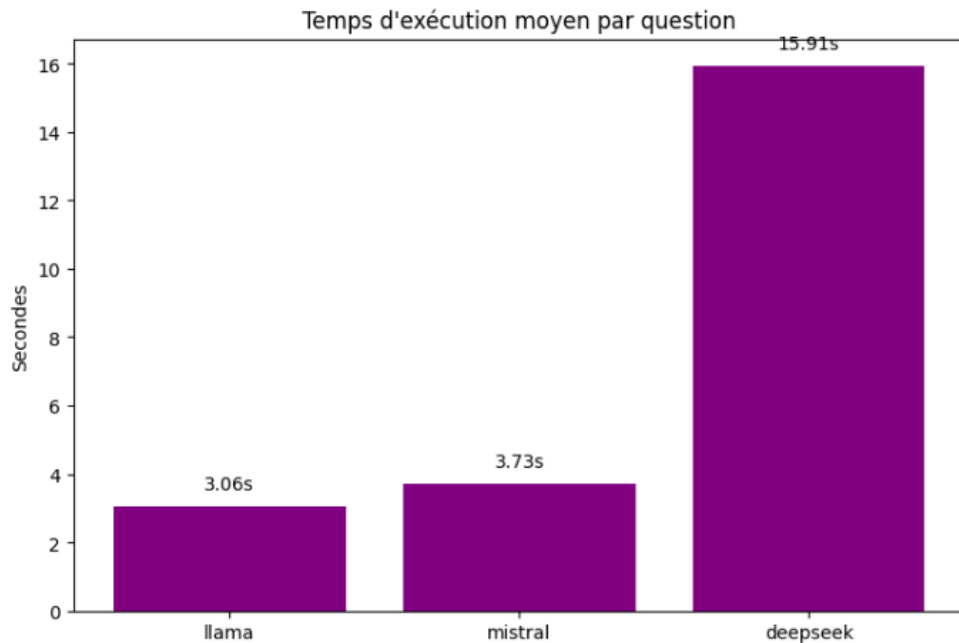


Figure 4 : Temps d'exécution moyen par question lors de l'évaluation GSM8K

Deepseek est largement plus lent que les deux autres, où en moyenne c'est presque 16s par question, c'est l'effet de l'étape de "thinking" qui le rend très lent, tandis que Mistral et Llama sont les plus rapide avec des temps similaires.

En comparant avec l'état de l'art (tableau suivant), on peut remarquer une légère différence, cela peut être causé par le fait que nous avons utilisé qu'un sous ensemble de 500 questions parmi les 8.5K questions au total dans le dataset, donc on peut observer une certaine variance.

Modèle Accuracy	Nos résultats	État de l'art
Mistral 7b	44.0%	52.2% [1]
LlaMA2 7b	5.0%	16.0% [1]
Deepseek r1	47.6%	61.9% [9]

Tableau 3 : comparaisons des résultats du test GSM8K avec l'état de l'art

5 Conclusion :

Sur l'ensemble des métriques évaluées, DeepSeek affiche la meilleure performance en termes d'accuracy, avec un score significativement supérieur aux autres modèles, Mistral se positionnant en second avec un écart relativement faible. Cependant, l'analyse qualitative révèle que DeepSeek rencontre des difficultés dans la structuration des réponses selon le format attendu, ce qui peut limiter son applicabilité dans des contextes où la précision de sortie est cruciale. Il peut aussi être limité par son temps d'inférence très élevé.

Mistral, bien qu'un peu moins performant en mathématiques, montre une meilleure cohérence dans la génération de réponses correctement formatées, ce qui traduit un compromis intéressant entre qualité des résultats numériques et respect du format de sortie, ainsi qu'un temps d'inférence très raisonnable.

Enfin, LLaMA souffre d'une faiblesse notable dans le traitement des tâches mathématiques, ce qui se traduit par une accuracy très faible, malgré une capacité à produire des réponses bien structurées. Cette observation suggère que ses faiblesses proviennent davantage d'un déficit dans la compréhension et le raisonnement mathématique que dans la génération textuelle formelle.

3.2.2 Raisonnement : HellaSwag

1. Description du Dataset

Le dataset utilisé pour cette évaluation est **HellaSwag** [4]. Il s'agit d'un benchmark conçu pour évaluer la capacité des modèles à effectuer du raisonnement de sens commun. Plus précisément, HellaSwag teste la complétion de phrases : étant donné un contexte initial (par exemple, le début d'une description d'activité ou d'un événement), le modèle doit choisir la fin la plus logique et naturelle parmi quatre options proposées. Ce dataset est particulièrement difficile car les options incorrectes sont souvent plausibles et stylistiquement similaires à la bonne réponse, piégeant ainsi les modèles qui se basent uniquement sur des similarités superficielles. Pour notre évaluation, nous avons utilisé la division "validation" du dataset, conformément aux pratiques courantes.

2. Méthodologie

L'objectif principal était d'évaluer la performance des trois modèles sur la tâche HellaSwag en utilisant une approche d'évaluation **0-shot (zero-shot)**.

- **Approche 0-Shot :** Dans cette configuration, les modèles sont évalués sans aucun exemple préalable de la tâche. Autrement dit, on ne leur montre pas de paires "contexte-bonne fin" avant de leur demander de résoudre de nouveaux exemples. Le modèle doit se baser uniquement sur sa connaissance générale et les instructions fournies pour accomplir la tâche.
- **Préparation des Données :** Pour chaque modèle, nous avons traité 500 questions (exemples) issues de l'ensemble de validation de HellaSwag. Chaque exemple se compose d'un contexte et de quatre fins possibles.
- **Interaction avec les Modèles :**
 - Un **prompt système** a été défini pour instruire clairement les modèles sur leur rôle et le format de réponse attendu. Il leur était demandé d'agir comme un assistant IA évaluant des complétions de phrases basées sur le raisonnement de sens commun. La consigne cruciale était de retourner **uniquement le chiffre (0, 1, 2 ou 3)** correspondant à l'index de la meilleure fin, sans aucune explication ou texte superflu.
 - Pour chaque exemple du dataset, un **prompt utilisateur** a été construit, présentant le contexte suivi des quatre fins numérotées (de 0 à 3). Le modèle devait alors choisir l'index de la fin la plus appropriée.

- Les requêtes ont été envoyées aux modèles via l'API d'Ollama, avec une température de 0.0 pour viser des réponses déterministes.
- **Extraction des Réponses** : Une fonction a été implémentée pour extraire l'index prédit (0, 1, 2 ou 3) de la réponse brute du modèle. Cette fonction recherche le dernier chiffre valide dans la réponse, permettant une certaine flexibilité si le modèle ne respectait pas strictement la consigne de ne retourner que l'index.

3. Calcul des Métriques de Performance

Le script évalue les performances de chaque modèle (LLaMA, Mistral et DeepSeek) en suivant une démarche en deux étapes :

3. 1. Sélection et traitement des prédictions

- Pour chaque question, le modèle renvoie un index censé correspondre à la bonne réponse (valeurs 0 à 3).
- Les prédictions non conformes (texte explicatif, index hors de l'intervalle [0,3]) sont automatiquement considérées comme incorrectes pour le comptage des réponses justes

3. 2. Calcul des métriques

- **Accuracy** : c'est le pourcentage de prédictions valides pour lesquelles l'index prédit coïncide exactement avec l'index de la réponse correcte.
- **F1-Score pondéré** : il s'agit de la moyenne harmonique entre précision et rappel, pondérée en fonction de la fréquence de chaque classe.

4. Analyse des Résultats

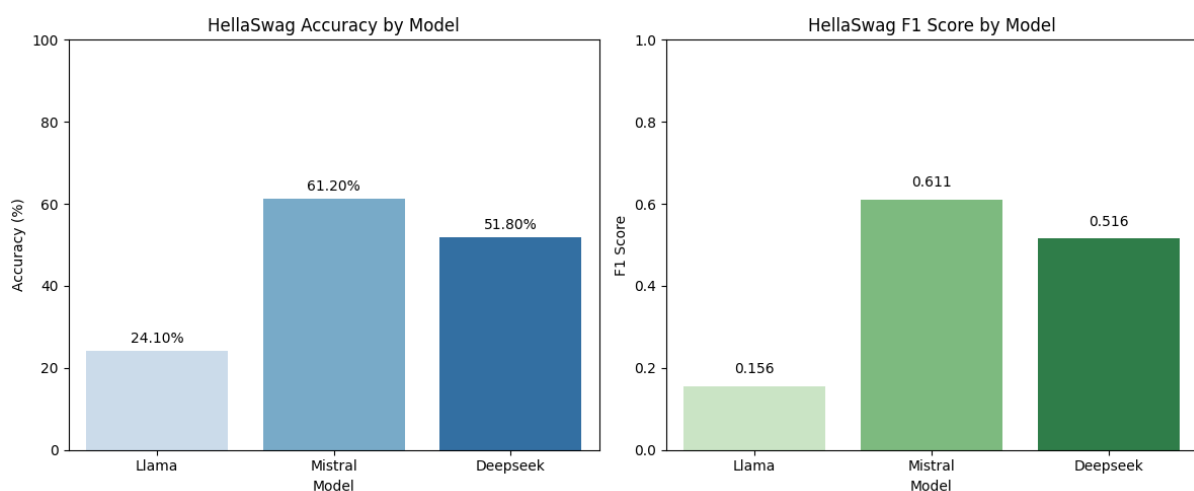


Figure 5 : Résultats de l'évaluation de HellaSwag.

- **Accuracy** :
 - **Mistral** a obtenu la meilleure accuracy avec environ **61.20%**.
 - **Deepseek** suit avec une accuracy d'environ **51.80%**.
 - **Llama** a montré une performance nettement inférieure, avec une accuracy d'environ **24.10%**. Une performance de 25% correspondrait à un choix

aléatoire parmi les quatre options. Llama se situe donc juste en dessous de ce seuil.

- **F1-Score :**

- Les tendances du F1-Score reflètent celles de l'accuracy.
- **Mistral** a également le F1-Score le plus élevé, autour de **0.611**.
- **Deepseek** obtient un F1-Score d'environ **0.516**.
- **Llama** a le F1-Score le plus bas, à **0.156**.

Ces résultats suggèrent que, dans une configuration 0-shot sur la tâche HellaSwag et pour les modèles testés :

- Mistral démontre une capacité de raisonnement de sens commun significativement meilleure que les deux autres modèles.
- Deepseek offre une performance respectable, surpassant Llama.
- Llama semble rencontrer des difficultés avec cette tâche sans exemples spécifiques, sa performance étant proche de l'aléatoire.

Efficacité en inférence :

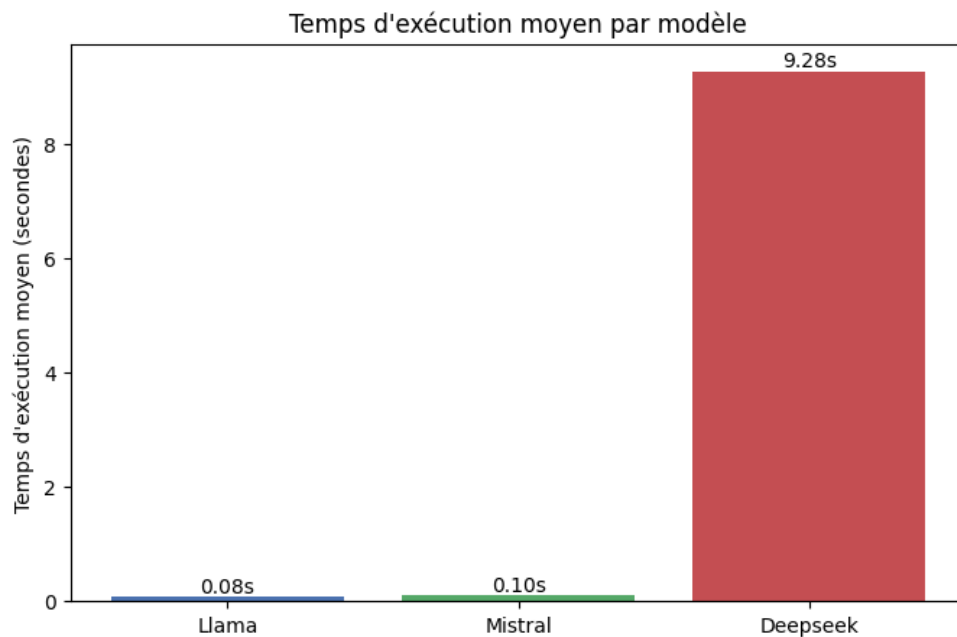


Figure 6 : Temps d'exécution moyen par question lors de l'évaluation HellaSwag.

D'une façon similaire au benchmark précédent, une différence très significative entre deepseek et les deux autres modèles. Deepseek tend toujours à avoir un temps très élevé, quant à mistral et llama, eux ils répondent extrêmement vite.

Faisant le lien avec l'état de l'art (tableau suivant), on observe un écart significatif cette fois ci, similairement avec le benchmark précédent, l'utilisation d'un sous ensemble des questions (500 pour un total de 10K ici) peut causer cette différence. Dans ce cas, c'est une marge suffisamment grande qu'on peut avoir une telle variance.

Modèle Accuracy	Nos résultats	État de l'art
Mistral 7b	61.2%	81.3% [1]
LlaMA2 7b	24.1%	77.1% [1]
Deepseek r1	51.8%	61.9% [9]

Tableau 4 : comparaisons des résultats du test HellaSwag avec l'état de l'art

5. Conclusion

L'évaluation en 0-shot sur HellaSwag a révélé des différences notables dans les capacités de raisonnement en sens commun des modèles testés. Mistral 7B s'est imposé comme le plus performant, suivi par DeepSeek R1. Les résultats de LLaMA 7B montrent que, sans exemples d'apprentissage spécifiques (few-shot), cette version peine à identifier la fin la plus logique parmi des options conçues pour être déroutantes. Bien que DeepSeek soit reconnu pour sa capacité de "thinking" prend tout le temps qu'il faut pour raisonner, il n'a pas surpassé Mistral, affichant une performance moyenne et éprouvant des difficultés à trouver la bonne réponse.

3.2.3 Génération de code : HumanEval :

1. Description du dataset :

Le jeu de données utilisé pour cette expérience est **HumanEval**, publié par OpenAI [2]. Il s'agit d'un benchmark reconnu dans la communauté de l'intelligence artificielle pour évaluer les capacités de génération de code Python par les modèles de langage. Il comporte 164 exemples, chaque exemple dans ce dataset comprend :

- Un **prompt** contenant la signature de la fonction à compléter ainsi que sa documentation (docstring).
- Une **solution canonique** servant de référence.
- Un **script de test** permettant de vérifier automatiquement la validité des réponses générées.
- Le **nom de la fonction** (entry point).

2. Méthodologie :

L'objectif de cette évaluation est de mesurer la capacité d'un modèle de langage à générer correctement du code Python fonctionnel en réponse à un prompt. Pour cela, nous avons suivi les étapes suivantes :

Préparation des prompts : Chaque prompt est utilisé seul (mode zero-shot).

Interaction avec le modèle : Pour chaque prompt, le modèle est sollicité via l'API Ollama, avec un prompt utilisateur et un prompt système très explicite, pour guider le modèle à fournir le résultat attendu.

Prompt système (system prompt) : il sert à définir le comportement attendu du modèle. Dans notre expérience, nous avons fourni un prompt système explicite précisant que la sortie devait uniquement contenir du code Python, sans commentaires ni explications supplémentaires. Cela vise à réduire l'ambiguïté et faciliter l'étape d'extraction du code.

Prompt utilisateur (user prompt) : c'est le contenu exact du prompt issu du dataset HumanEval, contenant la signature de la fonction, la docstring et une description de la tâche à effectuer pour encore insister. Il est injecté dynamiquement dans l'appel.

L'appel API inclut également un dictionnaire d'**options de génération**, destiné à contrôler le comportement du modèle :

- **temperature=0.1** : une température faible est utilisée pour rendre les générations plus **déterministes** [3]. Cela signifie que le modèle a tendance à choisir les tokens les plus probables, ce qui est crucial en génération de code où la cohérence syntaxique et logique est prioritaire.
- **top_p=0.9** : ce paramètre active le **nucleus sampling** [3], une stratégie de décodage qui limite la sélection aux tokens les plus probables dont la somme des probabilités cumulées atteint 90 %. Cela permet un équilibre entre **diversité et fiabilité** dans les réponses générées.

Extraction du code : Le code généré par le modèle est extrait à l'aide d'une expression régulière détectant les blocs de code Markdown.

Exécution contrôlée : Le code est ensuite testé en conjonction avec le script de test fourni par le dataset. Pour garantir la sécurité et l'isolement, chaque exécution est faite dans un sous-processus Python, avec une limite de temps (timeout) de 60 secondes.

Gestion des erreurs : Les erreurs d'exécution (assertions, erreurs de syntaxe, dépassement de temps) sont capturées pour évaluer la robustesse des réponses.

3. Calcul des métriques de performance :

Pour chaque tâche, une réussite est enregistrée si le code généré par le modèle passe avec succès tous les tests du script associé. La métrique principale est donc le **taux de réussite** (accuracy) qui mesure la proportion de problèmes pour lesquels le code exécuté passe sans erreur.

Nous utilisons également le **score F1** pour analyser plus finement les performances. Nous utilisons pour cela la fonction prédéfinie `f1_score` de la bibliothèque `sklearn.metrics`, qui permet d'automatiser ce calcul, où nous comparons le résultat du modèle (True s'il passe tous les tests, sinon False) avec un vecteur qui contient que des True.

4. Analyse des résultats obtenus :

Les graphiques ci-dessous comparent les performances des trois modèles sur les deux métriques : le **taux de réussite (accuracy)** et le **score F1**.

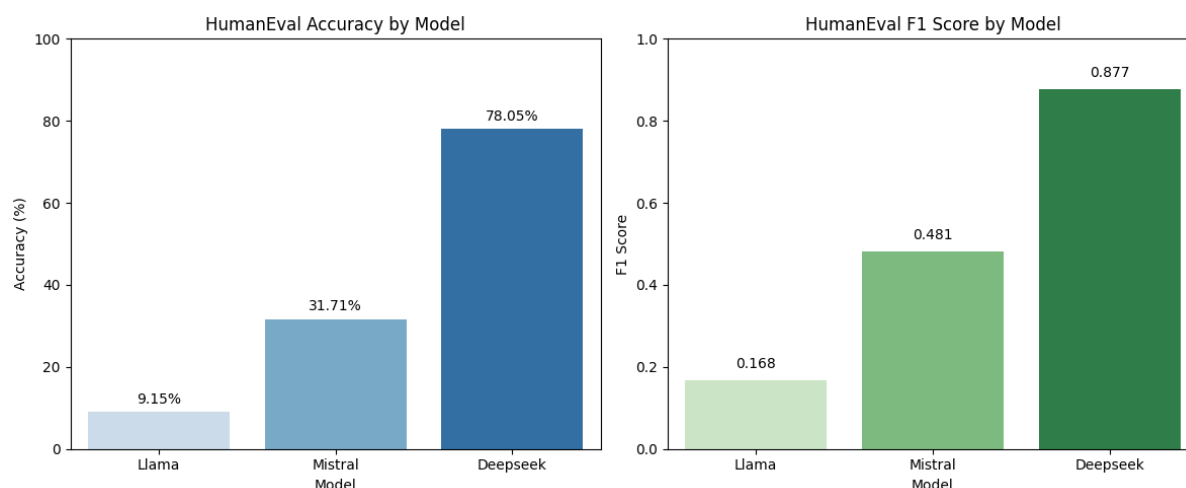


Figure 7 : Résultats de l'évaluation de HumanEval.

Accuracy :

- o **LLaMA** affiche une performance très faible avec seulement **9.15%** de taux de réussite, ce qui suggère une difficulté majeure à générer du code fonctionnel.
- o **Mistral** atteint **31.71%**, une nette amélioration par rapport à Llama, mais reste limité dans sa capacité à résoudre des tâches complexes.
- o **Deepseek** se distingue avec **78.05%**, une performance très élevée indiquant que le modèle est capable de comprendre et générer du code de manière fiable sur la majorité des tâches du benchmark HumanEval.

Score F1 :

- o **LLaMA** atteint seulement **0.1676**, confirmant une grande imprécision dans ses générations.
- o **Mistral** améliore ce score avec **0.4815**, mais reste à un niveau modéré.
- o **Deepseek** excelle avec **0.8767**, démontrant une capacité à produire des réponses non seulement correctes mais aussi précises et complètes.

Comparaison basée sur les types d'erreur :

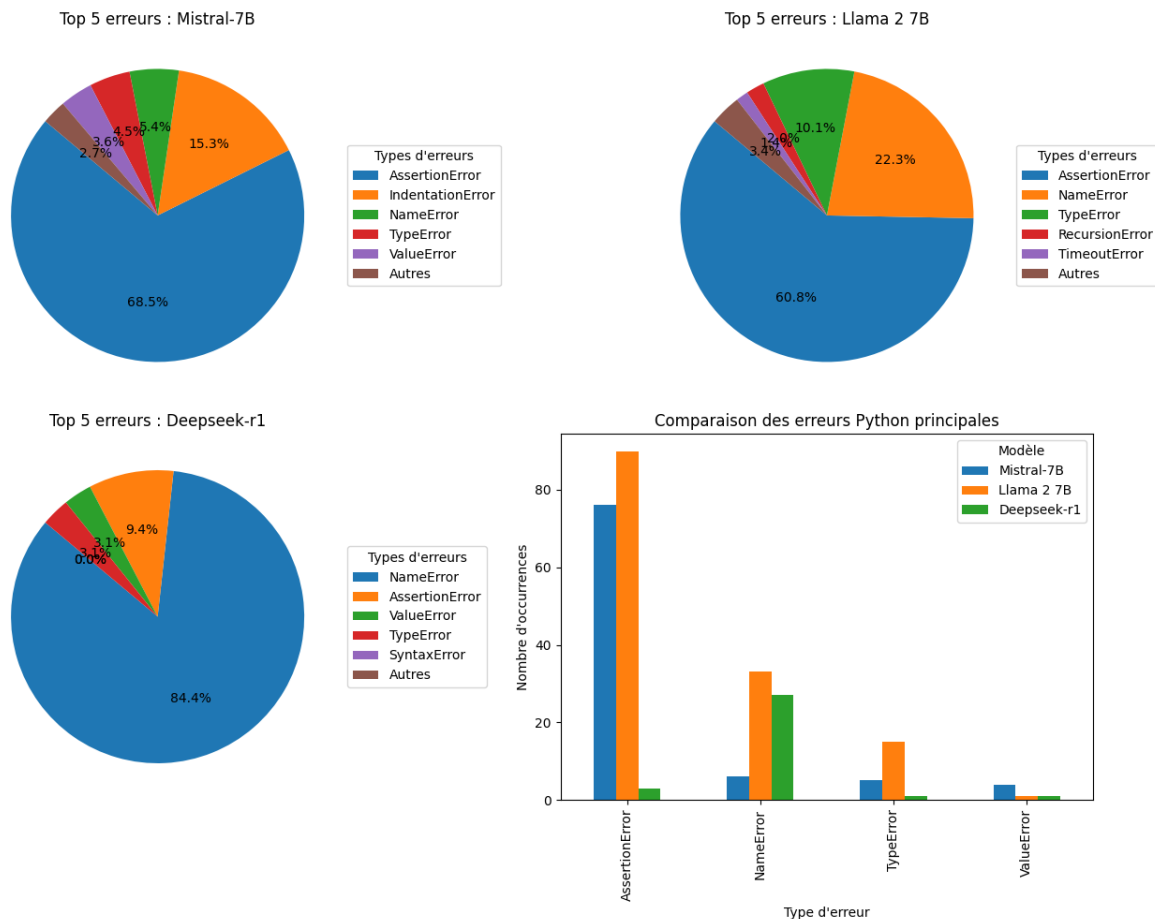


Figure 8 - visualisation comparative sur les types d'erreur python commises par chaque modèle

L'analyse de la visualisation en dessus révèle les points suivants :

- **Mistral-7B :**
 - Le type d'erreur dominant est l'**AssertionError**, représentant 68,5% de ses erreurs. Cela suggère que si le code généré par Mistral est souvent syntaxiquement correct, il ne répond pas aux exigences logiques des tâches.
 - La **IndentationError** est la deuxième plus fréquente (15,3%), indiquant des problèmes avec les indentations et la structure du code.
 - D'autres erreurs comme l'**NameError** (5,4%), la **TypeError** (4,5%) et la **ValueError** (3,6%) sont moins courantes.
- **Llama 2 7B :**
 - Similairement à Mistral, l'**AssertionError** est le type d'erreur le plus significatif (60,8%). Cela indique également des problèmes avec la justesse fonctionnelle du code généré.
 - La **NameError** (22,3%) est aussi très présente pour Llama, même beaucoup plus que pour Mistral.

- La **TypeError** est notablement plus élevée pour Llama (10,1%) que pour Mistral.
- Des erreurs de `TimeoutError` et `RecursionError` apparaissent également, bien que moins fréquemment.
- **Deepseek-r1 :**
 - L'**AssertionError** est de loin d'être le principal type d'erreur, constituant uniquement 9.4% de ses erreurs. Cela signifie que lorsque Deepseek arrive à donner un code sans problèmes de syntaxe ou d'exécution, ce code est généralement correct au sens du problème à résoudre et passe les tests avec succès.
 - La **NameError** (84,4%) est la plus dominante, donc la majorité des erreurs de DeepSeek sont reliés avec des variables ou des fonctions non définies.
 - D'autres erreurs comme la `ValueError` et la `TypeError` sont minimales.

Efficacité en inférence :

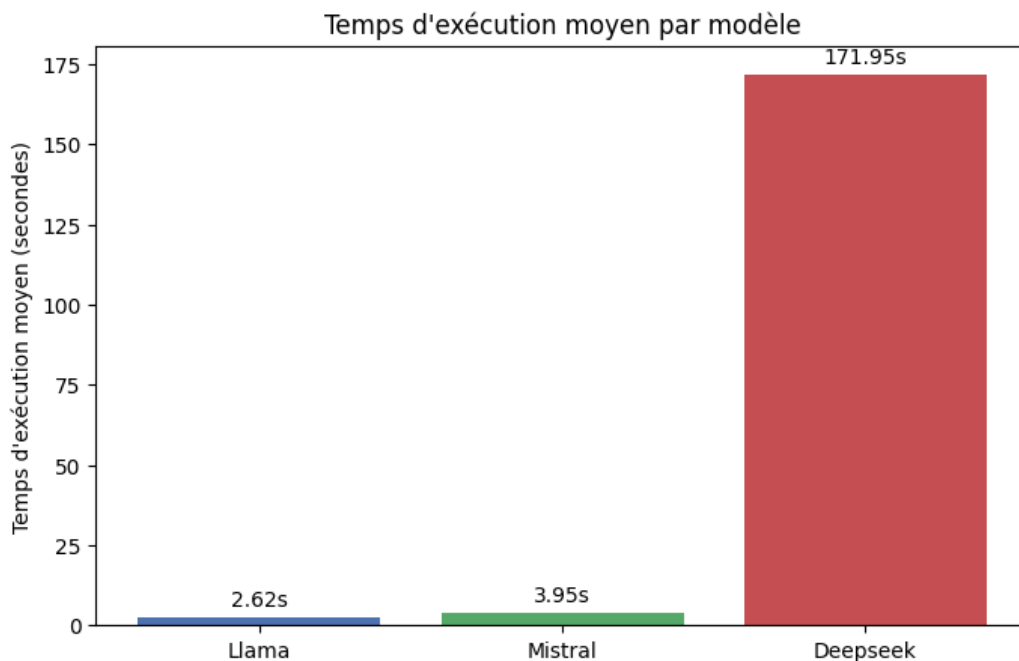


Figure 9 : Temps d'exécution moyen par question lors de l'évaluation HumanEval.

On peut voir clairement que deepseek prend énormément du temps à répondre, cela dû au fait qu'il prend son temps à réfléchir, c'est toujours le fameux "thinking", une étape cruciale pour lui pour bien comprendre la tâche mais, très coûteuse en temps d'inférence.

Interprétation globale :

Ces résultats mettent en évidence un écart significatif de performance entre les modèles. Deepseek surpasse largement LLaMA et Mistral aussi bien en termes de validité fonctionnelle (accuracy) qu'en qualité de réponse (F1). Cela suggère que Deepseek est bien plus adapté à la génération de code Python dans un contexte de test automatisé comme celui de HumanEval.

L'analyse des types d'erreurs affine cette interprétation, indiquant que ces modèles (LLaMA et Mistral) luttent non seulement avec la logique des problèmes, mais aussi avec la gestion correcte des variables, des noms de fonctions et des types de données. Llama 2 7B, en particulier, présente le plus grand nombre d'occurrences de **NameErrors** et **TypeError**s parmi les trois modèles, tandis que Mistral 7B souffre des erreurs d'indentation où le code n'est bien structuré, ce qui coïncide avec leurs taux de réussite global plus faible, mais les deux sont les plus performant en termes de rapidité.

En revanche, Deepseek-r1 voit ses erreurs dominées à 84,4 % par des **NameErrors**, ce qui indique que la majorité de ses échecs sont dus à l'utilisation de variables ou de fonctions non définies. Les **AssertionErrors** ne représentent que 9,4 % de ses erreurs, ce qui signifie que lorsque Deepseek-r1 parvient à générer un code sans erreur de syntaxe ou d'exécution, ce code est généralement correct sur le plan fonctionnel et passe les tests. Cette faible proportion d'erreurs liées à la logique métier, comparée aux deux autres modèles, suggère que Deepseek a une bonne compréhension des tâches à réaliser, mais qu'il rencontre principalement des difficultés liées à la définition ou à la gestion des éléments du code (comme les noms de variables ou de fonctions). Cependant, son temps d'inférence peut le rendre presque inutilisable si on augmente la complexité de la tâche à réaliser.

De plus, en comparant ces résultats avec l'état de l'art on trouve des scores similaires, on les résume dans le tableau suivant :

Modèle Accuracy	Nos résultats	État de l'art
Mistral 7b	31.71%	30.5% [1]
LlaMA2 7b	9.15%	11.6% [1]
Deepseek r1	78.05%	40.8% [9]

Tableau 5 : comparaisons des résultats du test HumanEval avec l'état de l'art

5. Conclusion :

Cette étude, basée sur le benchmark HumanEval, évalue la capacité des modèles à générer du code Python fonctionnel en zero-shot. Les résultats montrent des écarts significatifs : Deepseek atteint un taux de réussite de 78,05 %, contre 31,71 % pour Mistral et 9,15 % pour LLaMA.

L'analyse des erreurs révèle que, contrairement à LLaMA et Mistral, Deepseek échoue principalement à cause de **NameErrors** (84,4 %), des erreurs liées à des noms des variables ou des fonctions non définis, tandis que les **AssertionErrors** sont minoritaires (9,4 %). Cela montre que Deepseek maîtrise bien la sémantique et l'idée exposée dans le problème à résoudre, mais commet encore des erreurs liées à l'identification des variables ou des fonctions. Ces résultats confirment la supériorité de Deepseek pour la génération de code Python, tant en taux de réussite qu'en compréhension des tâches à réaliser.

4. Conclusion

Ce rapport a présenté une étude comparative des performances de trois grands modèles de langage open-source : LLaMA 7B, Mistral 7B et DeepSeek-R1-7B. Notre travail s'est concentré sur l'évaluation de ces modèles à travers trois benchmarks distincts : GSM8K pour le raisonnement arithmétique, HellaSwag pour le raisonnement de sens commun, et HumanEval pour la génération de code Python. Pour chaque benchmark, nous avons appliqué une méthodologie d'évaluation spécifique, en utilisant une approche 8-shots pour GSM8K, 0-shot pour HellaSwag, et 0-shot pour HumanEval, tout en exploitant un environnement Kaggle pour nos expérimentations. Les métriques d'accuracy et de F1-score ont été systématiquement employées pour quantifier les performances, complétées par une analyse des types d'erreurs pour l'évaluation sur HumanEval.

Les points forts de notre étude résident dans la mise en œuvre d'un protocole d'évaluation rigoureux sur des tâches variées, permettant de dégager des profils de performance distincts pour chaque modèle. Nous avons notamment observé la supériorité de DeepSeek en génération de code (78.05% d'accuracy sur HumanEval) et en raisonnement arithmétique en termes d'exactitude brute (47.6% sur GSM8K). Mistral s'est illustré par un meilleur raisonnement de sens commun sur HellaSwag (61.20% d'accuracy) et une meilleure consistance dans le formatage des réponses sur GSM8K. L'analyse des erreurs sur HumanEval a également fourni un éclairage pertinent sur la nature des difficultés rencontrées par chaque modèle.

Cependant, notre travail présente certaines limites. L'utilisation de sous-ensembles des datasets de test pour GSM8K (500 questions sur 8.5K) et HellaSwag (500 questions sur 10K) a pu introduire une variance dans nos résultats par rapport à l'état de l'art. De plus, bien que DeepSeek ait montré une forte exactitude sur GSM8K, ses difficultés à respecter le format de réponse attendu ainsi que son temps d'inférence très élevé constituent un inconvénient pratique. LLaMA, pour sa part, a affiché des performances globalement inférieures sur l'ensemble des tâches (par exemple, 5% d'accuracy sur GSM8K, 24.10% sur HellaSwag et 9.15% sur HumanEval).

Sur la base de nos résultats, nous pouvons faire une synthèse sur les cas d'utilisation des modèles :

- **LLaMA 7B** : Ce modèle a montré des performances globalement inférieures sur les trois benchmarks exigeants que sont le raisonnement mathématique, le raisonnement de sens commun et la génération de code. Bien qu'il soit rapide en inférence et capable de produire des réponses bien structurées, ses faiblesses dans la compréhension et le raisonnement le rendent moins adapté aux tâches complexes évaluées. Il pourrait être envisagé pour des applications moins critiques où la vitesse prime et où une adaptation (fine-tuning) conséquente est possible.
- **Mistral 7B** : Ce modèle s'est distingué par un excellent équilibre entre performances et efficacité. Il a obtenu les meilleurs résultats sur le benchmark HellaSwag, démontrant une solide capacité de raisonnement de sens commun. Ses

performances en mathématiques sur GSM8K étaient bonnes, se plaçant en deuxième position. Bien que moins performant que DeepSeek en génération de code, il reste compétent. Sa rapidité d'inférence et sa capacité à générer des réponses bien formatées en font un candidat idéal pour des applications interactives nécessitant une bonne compréhension du langage, du raisonnement général et une réactivité appréciable, telles que les assistants virtuels ou les outils d'aide à la rédaction.

- **DeepSeek-R1-7B** : Ce modèle excelle dans les tâches nécessitant un raisonnement profond et spécialisé. Il a dominé le benchmark HumanEval pour la génération de code, avec une grande capacité à comprendre la logique des problèmes, malgré des erreurs principalement liées à la gestion des noms de variables ou fonctions. Il a également obtenu la meilleure accuracy sur GSM8K pour le raisonnement mathématique. Son principal inconvénient est son temps d'inférence significativement plus long. DeepSeek est donc particulièrement recommandé pour des tâches complexes et spécialisées comme la génération et l'analyse de code avancées ou la résolution de problèmes mathématiques, où la précision prime sur la vitesse d'exécution et où des mécanismes de post-traitement peuvent pallier d'éventuels problèmes de formatage.

Plusieurs pistes pourraient être explorées pour approfondir ce travail. Il serait pertinent de réévaluer les modèles sur l'intégralité des benchmarks afin de consolider les comparaisons avec l'état de l'art. Des investigations supplémentaires pourraient porter sur l'optimisation de l'inférence de DeepSeek, par exemple via des techniques de distillation ou de quantification plus poussées, pour réduire sa latence. L'étude pourrait également être étendue à d'autres benchmarks couvrant des aspects différents du langage (ex : traduction, résumé de texte) ou à des modèles de tailles différentes. Enfin, une analyse plus qualitative des "échecs" et des "réussites" de chaque modèle pourrait offrir des aperçus plus fins sur leurs mécanismes internes de raisonnement et de génération.

5. Bibliographie :

- [1] Jiang, A.Q. et al. 2023. Mistral 7B. CoRR. abs/2310.06825. DOI : <https://doi.org/10.48550/ARXIV.2310.06825>.
- [2] Chen, M. et al. 2021. Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374.
- [3] Holtzman, A. et al. 2020. The Curious Case of Neural Text Degeneration. In International Conference on Learning Representations (ICLR).
- [4] Zellers, Rowan, et al. "Hellaswag: Can a machine really finish your sentence?." *arXiv preprint arXiv:1905.07830* (2019).
- [5] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). *Training Verifiers to Solve Math Word Problems*. arXiv preprint arXiv:2110.14168.
- [6] DeepSeek-AI, Guo, D., Yang, D., Zhang, H., et al. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv preprint arXiv:2501.12948.
- [7] Vaswani, A. et al. (2017). *Attention Is All You Need*. arXiv preprint arXiv:1706.03762.
- [8] Touvron, H., et al. (2023). *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971.
- [9] Hugging Face – RedHatAI. *Page du modèle DeepSeek-R1-Distill-Qwen-7B-quantized.w8a8*. Consulté depuis <https://huggingface.co/RedHatAI/DeepSeek-R1-Distill-Qwen-7B-quantized.w8a8>